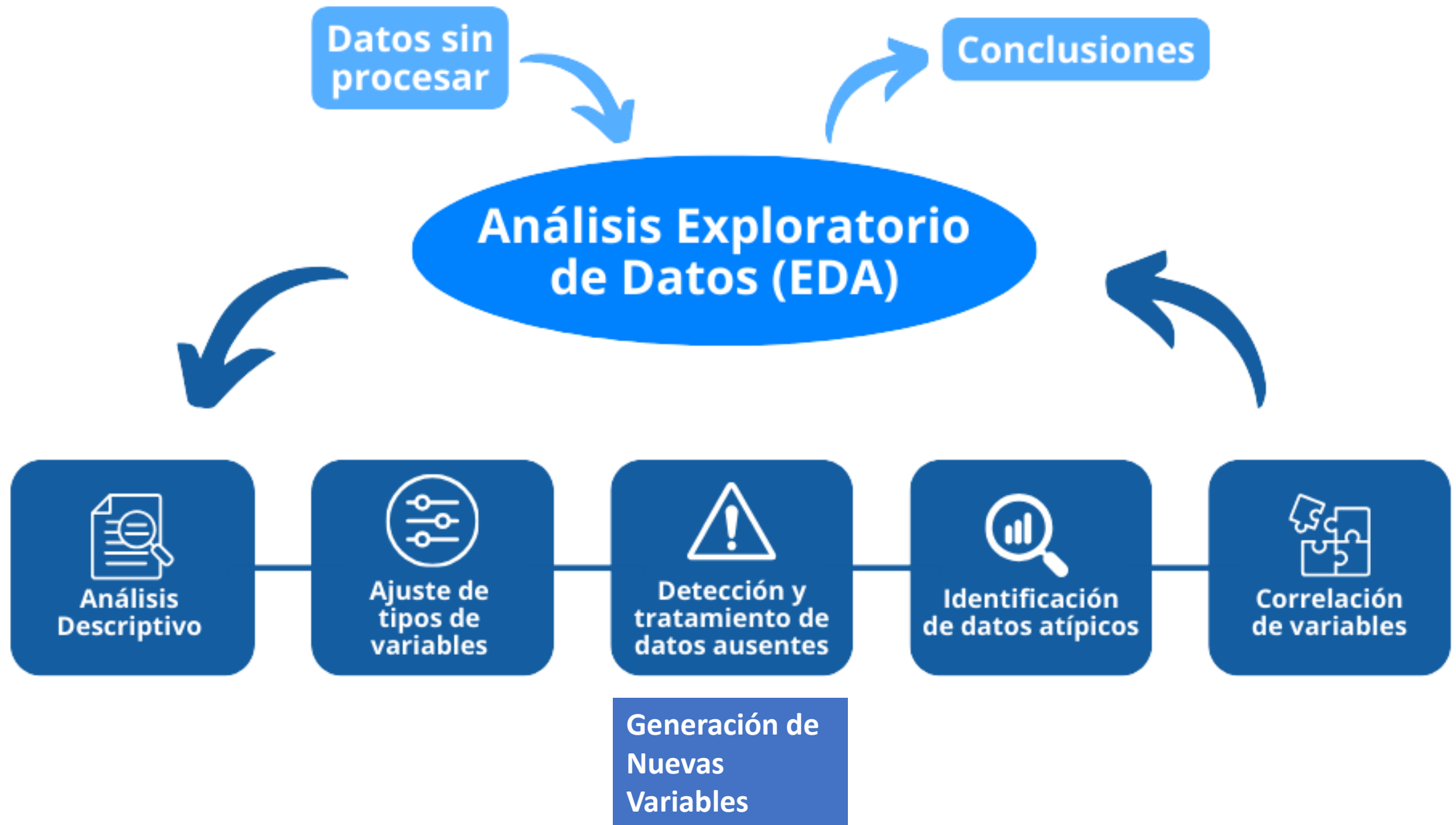




Correlación de Variables:

¿Qué es?

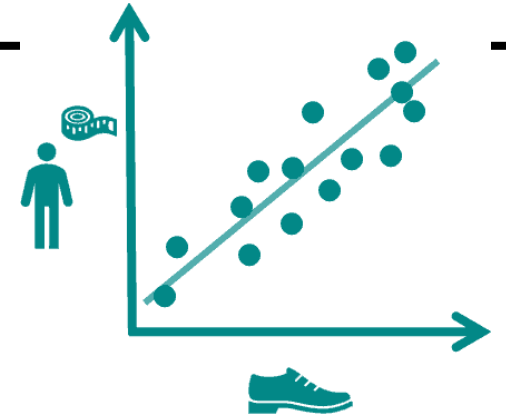
Analizar la relación entre dos o más variables

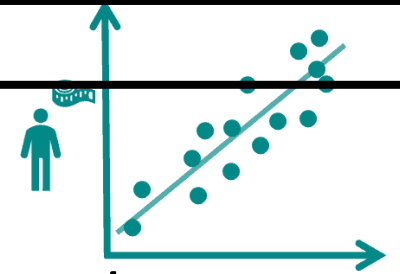
¿Por qué es necesario?

Para descartar posibles variables que aporten información redundante en el conjunto de datos, ocasionando ruido en los análisis o para detectar posibles relaciones de causalidad con variable objetivo

Tratamiento

Calcular los coeficientes de correlación para las variables para detectar coeficientes cercanos a 1 o -1.





Correlación de Variables:

Puedes armar mapas de calor, que reúnen en un solo gráfico todas las correlaciones de a pares

Comparar dos variables numéricas. Podemos usar una gráfica de dispersión, donde cada punto es representado por un caso (fila)

Comparar una variable numérica con una variable categórica y usar por ejemplo un boxplot para cada valor categórico

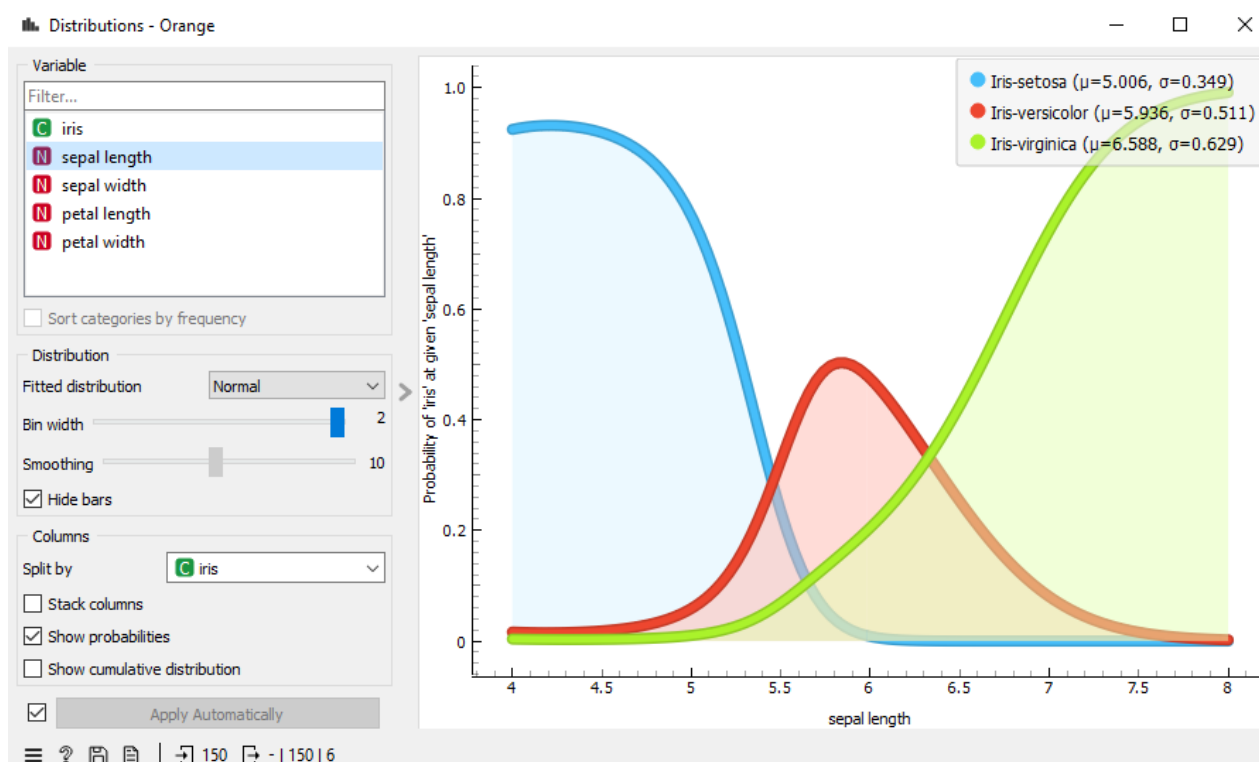
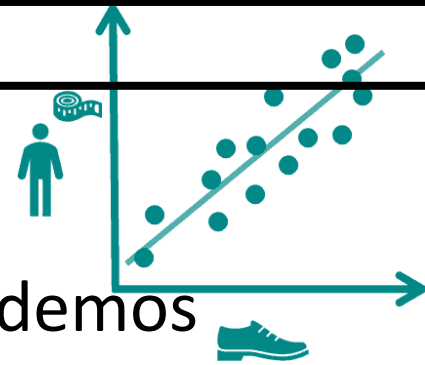
Comparar dos variables categóricas usando gráficos de barras apiladas



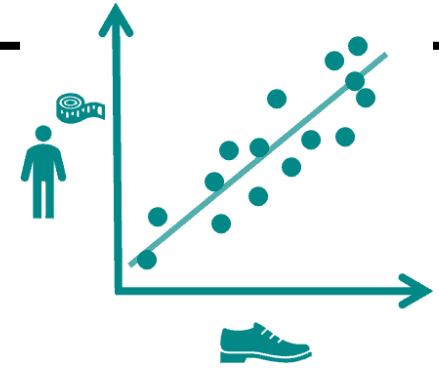
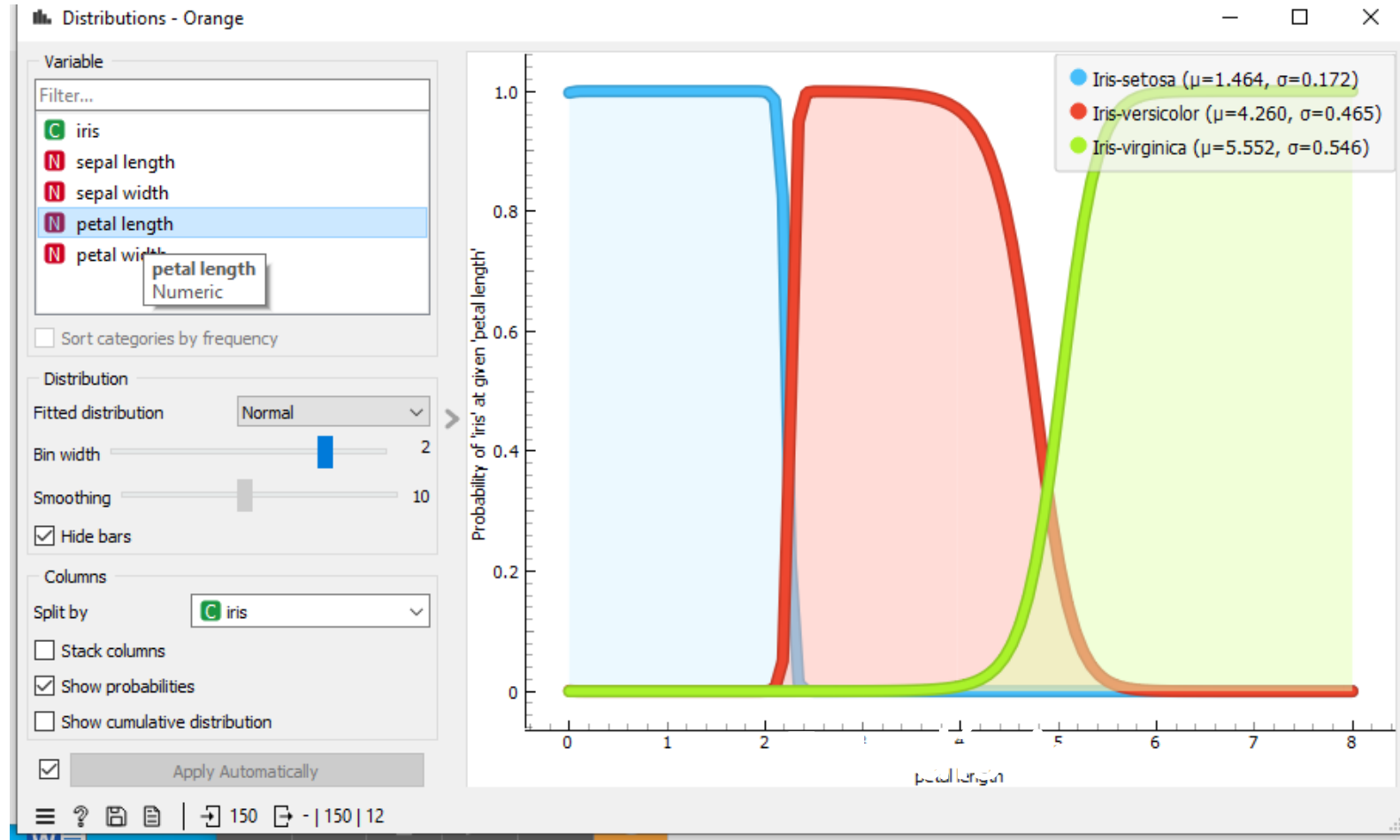
Mirá otra idea en la próxima diapo

Correlación de Variables:

Incluso, si las dos series toman valores en intervalos parecidos, podemos simplemente vincularlas en un único gráfico y observar cómo evolucionan



Correlación de Variables:





Agregado de Variables:

¿Qué es?

Definir una nueva columna de datos a partir de otras o de datos externos; o ambas cosas

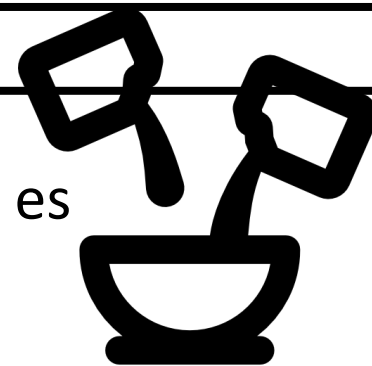
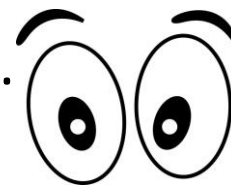
¿Por qué es necesario?

Para obtener mayor o mejor información que pueda ser de utilidad al Modelo

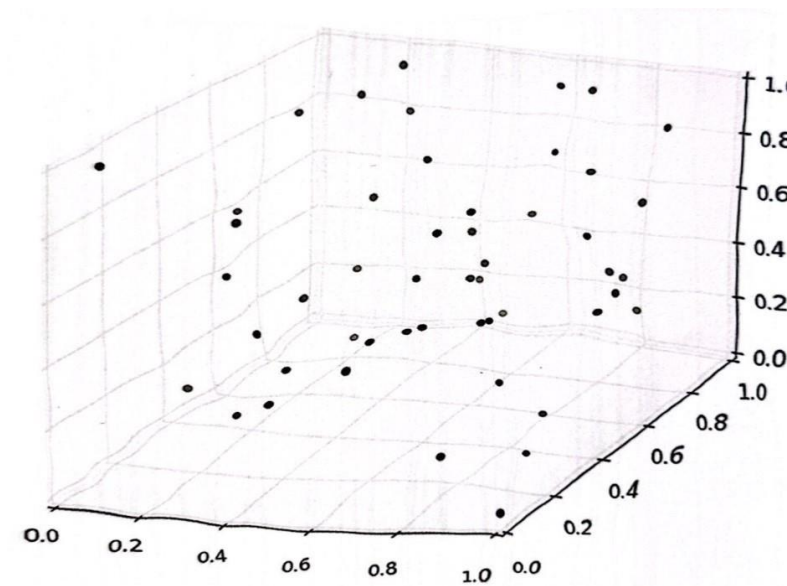
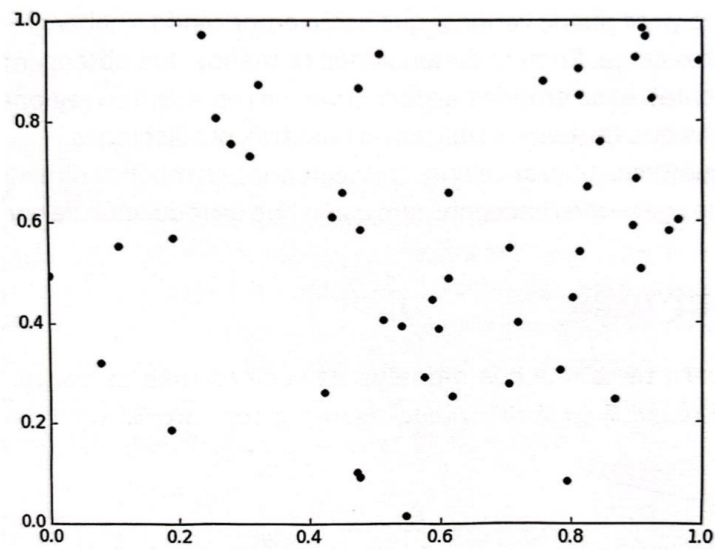
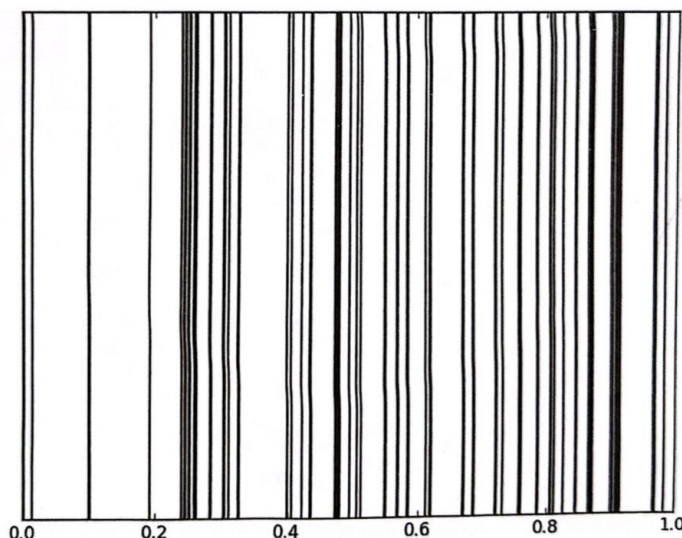
Tratamiento

- ✓ Se pueden agregar columnas desde otros Dataframes o Fuentes Externas de Datos
- ✓ Se pueden agregar datos constantes externos y aplicarlos en alguna reformulación de una columna original (pago en u\$s, si tengo cotización del u\$s de esa fecha puedo generar pago en \$)
- ✓ Se puede obtener nuevas columnas a partir de reformular (armar una función) otras originales

Uno de los motivos habituales para agregar columnas/características/variables es **disminuir** la cantidad de columnas a enviar al Modelo ..



La Maldición de la Dimensionalidad





Análisis de Componentes Principales (PCA)

Método estadístico utilizado para la reducción de la dimensionalidad de un Dataset conservando su información

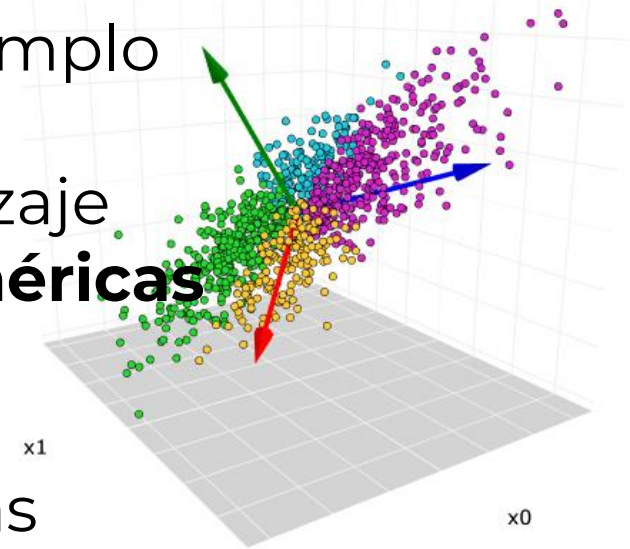
No es el único sistema. Existe el método de **Selección Inteligente de Paso a Paso (stepwise selection)**, por ejemplo

La técnica **PCA** forma parte de los algoritmos de aprendizaje no supervisado de ML, y se utiliza solo con **variables numéricas**

Es muy sensible a outliers!!

¿Para qué la usamos?

Para reducir dimensionalidad y poder armar modelos más eficientes con regresión lineal, regresión logística, K-NN, K-means, etc. La idea es generar **Super Columns**
Menos tiempo de proceso y mejores resultados



Análisis de Componentes Principales (PCA)

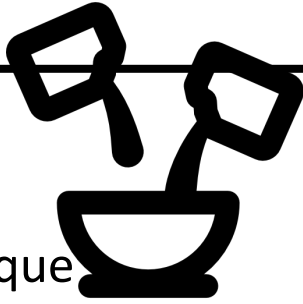
Para entender la idea subyacente consideremos un edificio con 50 departamentos
¿Te imaginas una Reunión de Consorcio?



En todo Consorcio siempre están los propietarios con **opinión propia** sobre cada tema y los **Seguidores Seriales**

Para ser más prácticos, el Administrador identifica 8 propietarios que parecen ser los líderes de 8 grupos con opiniones diversas y les pide a cada uno de ellos que averigüen lo que opinan los restantes vecinos sobre los temas a tratar (seguro cada uno de ellos, aunque recojan honestamente las opiniones, las incorporarán de manera sesgada; no importa, todos los sesgos participan)

Al final del día, para el Administrador será mucho más sencillo y efectivo tanto organizar la reunión con los 8 líderes/voceros, como tomar las decisiones. Aunque haya habido un trabajo previo adicional y el resto de los propietarios no participen de la reunión, su visión está representada. No se pierden opiniones



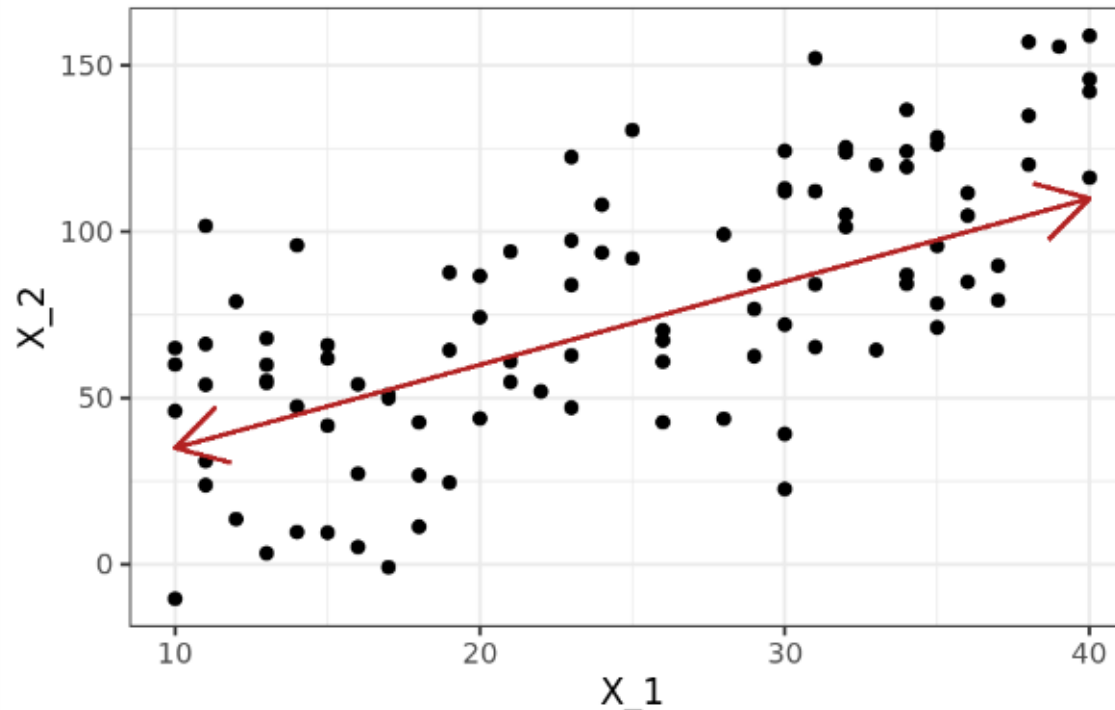
Análisis de Componentes Principales

Dado un Dataset con n filas, cada una con p variables (columnas) ($X_1, X_2, \dots, X_p$)

PCA permite encontrar un número z de factores (**Componentes Principales**) subyacentes (con $z < p$) que expliquen aproximadamente lo mismo que las p variables originales

Necesitamos todas las variables (características, columnas) originales para calcular las **Componentes**

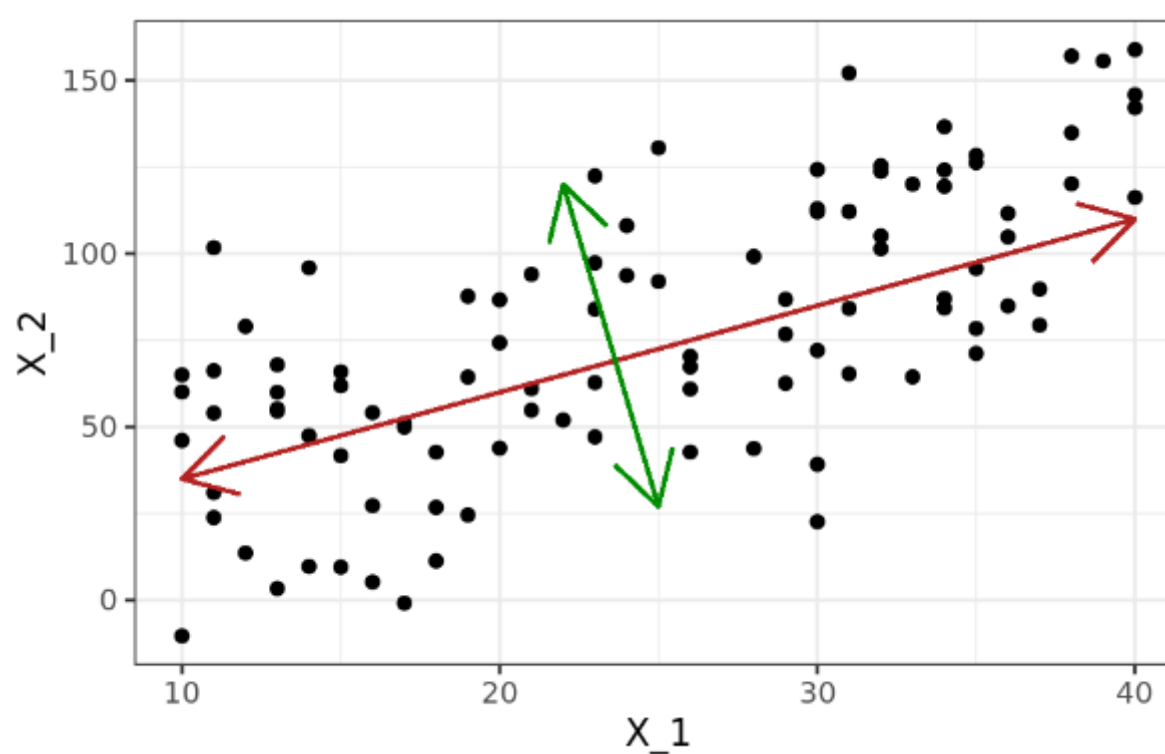
¿Cuál es la idea?



Capturar en una Componente Principal, que es una columna sintética creada por el algoritmo, la mayor varianza posible de valores (opiniones), para no perder el valor de la información original, aunque comprimamos esas opiniones en columnas que las sintetizan (el líder del grupo con más seguidores con opiniones diferentes)



Análisis de Componentes Principales



Una vez que identifiqué al capo del grupo más heterogéneo voy por el líder del segundo, y así ...

En el gráfico está representado un modelo bidimensional, la nube está en el plano y consideramos la correlación de sólo dos variables (columnas). Si en realidad tenemos un espacio n -dimensional, con n columnas, habrá flechas (vectores de dirección) para todos lados ...



Análisis de Componentes Principales

¿Cómo se calcula?

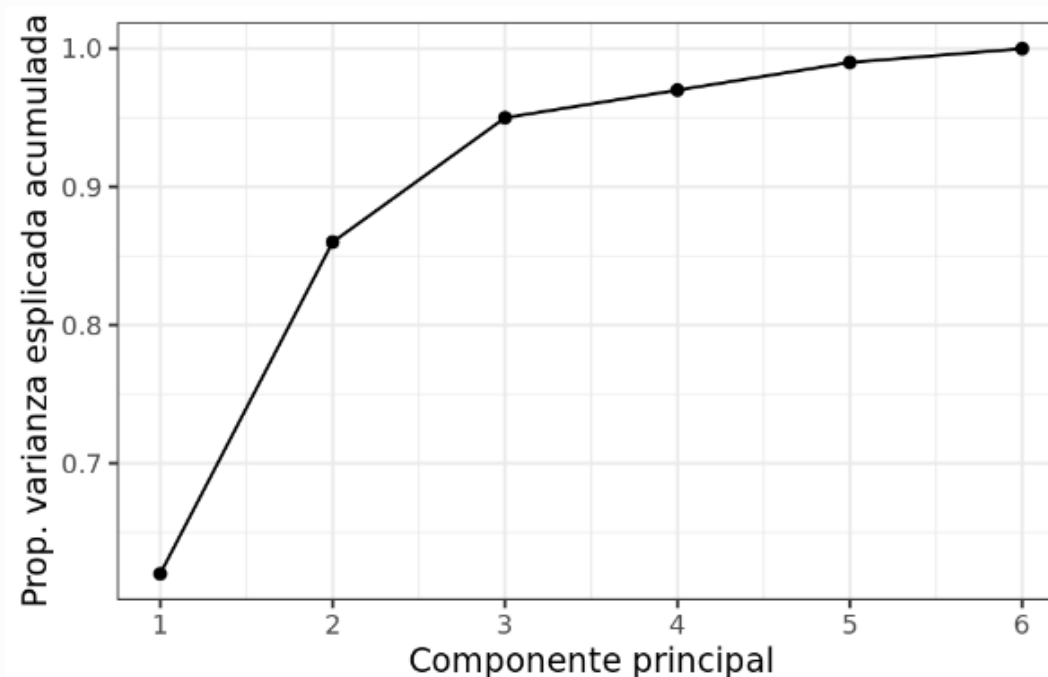
En general se buscan los autovectores (eigenvectors) de la matriz cuadrada de $k \times k$ donde $k = \min(p, n-1)$ y se toma como PC1 el de mayor autovalor

Luego se ordena el próximo

Esto permite que la PC1 explique la mayor varianza, la segunda no esté correlacionada a la primera (vector ortogonal) y tenga la segunda mayor varianza, y así ...

¿Cuántas PC usar?

En general se busca que la varianza total explicada alcance un mínimo propuesto





Análisis de Componentes Principales

¿Qué es un autovector o eigenvector o vector propio?

Los eigenvectors son un caso particular de multiplicación entre una matriz cuadrada y un vector. Donde el vector resultante de la multiplicación es un múltiplo entero del vector original. Los autovectores de una matriz son todos aquellos vectores que, al multiplicarlos por dicha matriz, resultan en el mismo vector o en un múltiplo entero del mismo. Son todos ortogonales (perpendiculares) entre si

El factor multiplicativo que se le aplica al autovector para que de el vector resultante es el autovalor (eigenvalue)

4 en el ejemplo

$$\begin{pmatrix} 2 & 3 \\ 2 & 1 \end{pmatrix} \times \begin{pmatrix} 3 \\ 2 \end{pmatrix} = \begin{pmatrix} 12 \\ 8 \end{pmatrix} = 4 \times \begin{pmatrix} 3 \\ 2 \end{pmatrix}$$

Otro eigenvector arriba: (1,-1)

$M \times N * N \rightarrow M$

Los elementos de la primera fila, uno a uno, se multiplican por los elementos del vector y la suma de los productos es el primer elemento del vector resultante. Y así...



Análisis de Componentes Principales autovalor

$$\begin{bmatrix} 2 & 3 \\ 2 & 1 \end{bmatrix} \times \begin{bmatrix} 1 \\ -1 \end{bmatrix} = -1 \begin{bmatrix} -1 \\ 1 \end{bmatrix}$$

Otros Métodos de Reducción de la Dimensionalidad

SNE y t-SNE, Sammon Mapping, Isomap, LLE, CCA, MVU, Laplacian Eigenmaps

$$\begin{pmatrix} 2 & 3 \\ 2 & 1 \end{pmatrix} \times \begin{pmatrix} 3 \\ 2 \end{pmatrix} = \begin{pmatrix} 12 \\ 8 \end{pmatrix} = 4 \times \begin{pmatrix} 3 \\ 2 \end{pmatrix}$$

Otro eigenvector arriba: (1,-1)

$M \times N * N \rightarrow M$

Los elementos de la primera fila, uno a uno, se multiplican por los elementos del vector y la suma de los productos es el primer elemento del vector resultante. Y así...



t-SNE y SNE (Stochastic Neighbor Embedding, Integración de Vecinos Estocásticos)

¿Para qué se usa?

Reducción de Dimensionalidad

¿Cuál es la idea?

Preservar la misma cercanía o distancia entre dos datos en el espacio de n dimensiones cuando reducimos a m (2 o 3 max) dimensiones

¿Pará qué es bueno?

- ✓ Para permitir representaciones gráficas de correlación de 2 variables (características) o 3 que muestren con claridad grupos, patrones o outliers
- ✓ Para clustering e identificación de subgrupos no evidentes a prima facie con muchas columnas

¿Cuál es la idea?

Asocia probabilidades a las distancias en cada dimensión entre dos puntos e intenta reproducir esa probabilidad de vecino en el espacio reducido





t-SNE y SNE (Stochastic Neighbor Embedding, Integración de Vecinos Estocásticos)

Algoritmo

1. Calcula similitudes en alta dimensión

Para cada par de puntos en el espacio original calcula qué tan parecidos son. Convirtiendo las distancias entre puntos en probabilidades: Si dos puntos están muy cerca, tienen una alta probabilidad de ser **vecinos**, cuánto más lejanos, menor probabilidad

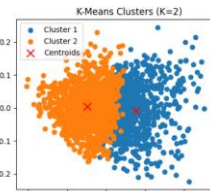
2. Hace lo mismo en baja dimensión (2D o 3D)

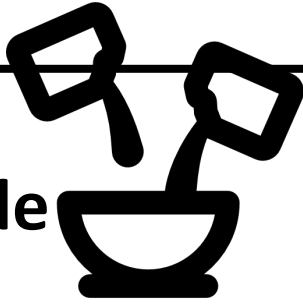
Coloca los puntos en el nuevo espacio (2D) al azar al principio, y ajusta sus posiciones poco a poco. Calcula nuevamente las probabilidades de vecindad entre puntos, pero ahora en 2D

3. Compara los dos espacios

Compara las probabilidades en el espacio original con las probabilidades en el espacio reducido; y trata de hacer que se parezcan lo más posible. Para eso, mueve los puntos en 2D (como si jugara con imanes) hasta que las relaciones de vecindad sean similares

Usa una fórmula llamada Kullback-Leibler Divergence para medir la diferencia entre las probabilidades y t-SNE usa una distribución llamada t-Student en 2D, que ayuda a evitar que los grupos se aplasten; esa es la mejora con respecto a SNE, que es anterior



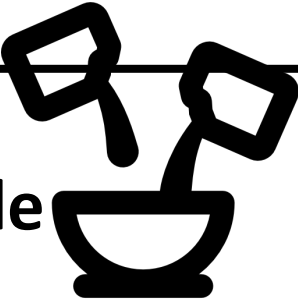


t-SNE y SNE (Stochastic Neighbor Embedding, Agrupación Estocástica de Vecinos)

```
from sklearn.manifold import TSNE
from sklearn.datasets import load_digits
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Cargamos un dataset de ejemplo: imágenes de dígitos (0-9)
digitos = load_digits()
X = digitos.data          # características (64 por imagen)
y = digitos.target        # etiquetas (0 a 9)
```

```
# Reducimos las dimensiones de 64 a 2 usando t-SNE
tsne = TSNE(n_components=2, random_state=42, perplexity=30)
XtSne = tsne.fit_transform(X)
```



t-SNE y SNE (Stochastic Neighbor Embedding, Agrupación Estocástica de Vecinos)

Para qué se usan `random_state` y `perplexity`?

`random_state`: es una semilla aleatoria que controla la aleatoriedad interna del algoritmo (permite seleccionar el par de función de generación de números pseudo aleatorios)
¿Para qué se usa? Para lograr previsibilidad. Si el conjunto de datos no tiene límites de grupos bien marcados seguramente la distribución de los casos en los grupos variará dependiendo de la asignación aleatoria de grupos inicial. Es decir, corro el mismo algoritmo varias veces y obtengo diferentes resultados. Si quiero previsibilidad fijo la semilla y eventualmente puedo comparar con los resultados que arrojaría otra



t-SNE y SNE (Stochastic Neighbor Embedding, Agrupación Estocástica de Vecinos)

Perplexity: es un parámetro que controla qué tan **local** o **global** es la vista que t-SNE tiene de los datos. Le dice al algoritmo **cuántos vecinos cercanos** considerar para cada punto al calcular las relaciones de similitud (igual que el K de K-NN)

Intuitivamente, una **perplexity**:

5–10, el algoritmo se enfoca más en estructuras locales (pequeños grupos o clústeres)

30–50, el algoritmo mira estructuras más globales (relaciones entre grupos)

Tip:

- ✓ Valores comunes: entre 5 y 50
- ✓ Si son pocos datos, usá valores más bajos (por ejemplo 5–30)
- ✓ Si son muchos datos, podés aumentar un poco

En el ejemplo de los dígitos una **perplexity** > 30 tendería a presentar menos de 10 grupos. Por ahí los dígitos más redonditos juntos, etc.



t-SNE y SNE (Stochastic Neighbor Embedding, Integración de Vecinos Estocásticos)

```
# Graficamos los resultados
```

```
plt.figure(figsize=(10, 6))
```

```
sns.scatterplot(x=XtSne[:, 0], y=XtSne[:, 1], hue=y, palette='tab10',  
legend='full')
```

```
plt.title("Visualización de dígitos usando t-SNE")
```

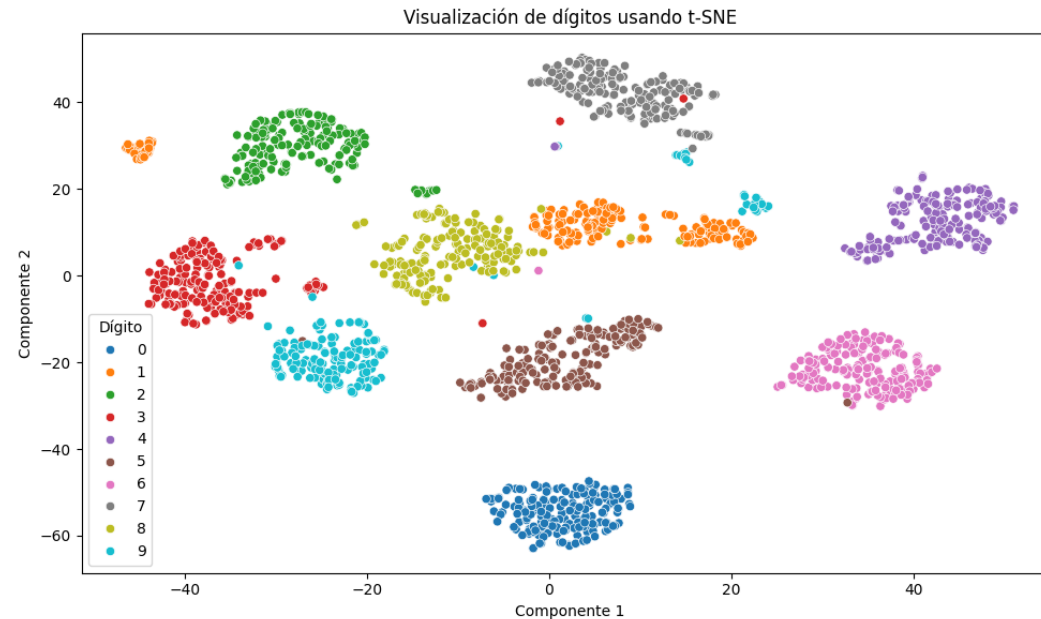
```
plt.xlabel("Componente 1")
```

```
plt.ylabel("Componente 2")
```

```
plt.legend(title='Dígito')
```

```
plt.tight_layout()
```

```
plt.show()
```

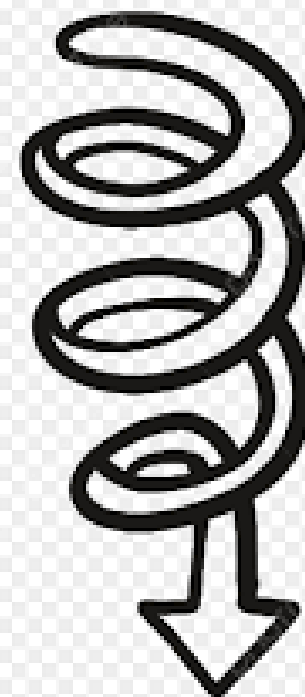


Cada gráfico que generamos y cada estadística descriptiva que calculamos nos lleva a **nuevas preguntas**, refinando y profundizando nuestro análisis

Durante todo este proceso no avanzamos en una sola dirección, sino que **nos movemos en espirales**, volviendo sobre nuestros pasos con nuevos conocimientos y preguntas

Este ciclo de visualizar, transformar, modelar y luego visualizar nuevamente es la magia del EDA y lo que nos permite explorar los datos de una manera más exhaustiva

Con cada iteración, nuestro entendimiento de los datos se profundiza, y emergen *insights* más claros y detallados



REGLA DE ORO!! No modificar sobre los datos originales!!

Agregar columnas